fig_schemes.py — 이 문서의 도식 그림

```python
"""Schematic figures of the methods (no computed results): spatial operators, one certified step, orders of discretization."""
import numpy as np
import matplotlib.pyplot as plt
from viz import check_clip, BLUE, ORANGE, AQUA, GRAY, RED

# ---------------------------------------------------------------- figure 1: spatial operators and one certified step
fig, ax = plt.subplots(1, 2, figsize=(7.2, 2.9), gridspec_kw=dict(width_ratios=[1.05, 1], wspace=0.28, left=0.03, right=0.98, bottom=0.17, top=0.86))
a = ax[0]; xs = np.linspace(0, 1, 6); h = xs[1] - xs[0]
rows = {"FDM (꼭짓점)": 2.2, "FVM (칸 중심)": 1.2, "FEM P1 (요소)": 0.2}
for name, y in rows.items():
    a.plot([0, 1], [y, y], color=GRAY, lw=1)
    tt = a.text(0, y + 0.62, name, fontsize=7.5, fontweight="bold", va="bottom", ha="left"); tt._ide_key = True
y = rows["FDM (꼭짓점)"]; a.plot(xs, np.full(6, y), "o", color=BLUE, ms=5)
a.annotate("", xy=(xs[2] + h, y + 0.28), xytext=(xs[2] - h, y + 0.28), arrowprops=dict(arrowstyle="-", color=BLUE, lw=0.8))
a.text(xs[2], y + 0.32, "(1, -2, 1)/h²", fontsize=6.3, ha="center", va="bottom", color=BLUE)
y = rows["FVM (칸 중심)"]; xc = np.linspace(0.1, 0.9, 5)
for e in np.linspace(0, 1, 6): a.plot([e, e], [y - 0.18, y + 0.18], color=ORANGE, lw=1)
a.plot(xc, np.full(5, y), "s", color=ORANGE, ms=4)
a.annotate("", xy=(0.6, y + 0.3), xytext=(0.4, y + 0.3), arrowprops=dict(arrowstyle="->", color=ORANGE, lw=0.8))
a.text(0.5, y + 0.34, "면 플럭스", fontsize=6.3, ha="center", va="bottom", color=ORANGE)
y = rows["FEM P1 (요소)"]; xf = np.linspace(0, 1, 401)
for i, xi in enumerate(xs):
    hat = np.maximum(0, 1 - np.abs(xf - xi) / h) * 0.45; on = hat > 0
    a.plot(xf[on], y + hat[on], color=AQUA, lw=0.9 if i != 2 else 1.6)
a.plot(xs, np.full(6, y), "o", color=AQUA, ms=4)
a.set_xlim(-0.04, 1.04); a.set_ylim(-0.1, 3.15); a.axis("off")
tt = a.text(0, 3.12, "(가) 공간 연산자: 모두 (L, w)", transform=a.transData, fontsize=8.5, fontweight="bold", va="bottom"); tt._ide_key = True

a = ax[1]; hh = 1.0; q = 8
sg = np.linspace(0, hh, 300); p = 0.3 + 0.9 * sg - 1.4 * sg ** 2 + 0.75 * sg ** 3
a.plot(sg, p, color=BLUE, lw=1.8, label="후보 p(t)")
for j in range(q + 1): a.axvline(j * hh / q, color=GRAY, lw=0.5)
mids = (np.arange(q) + 0.5) * hh / q; pm = 0.3 + 0.9 * mids - 1.4 * mids ** 2 + 0.75 * mids ** 3
a.plot(mids, pm, "o", color=ORANGE, ms=3.5, label="동결점(하위 걸음 중점)")
for j in range(q):
    s0, s1 = j * hh / q, (j + 1) * hh / q; ss = np.linspace(s0, s1, 20); pp = 0.3 + 0.9 * ss - 1.4 * ss ** 2 + 0.75 * ss ** 3
    a.add_patch(plt.Rectangle((s0, pp.min() - 0.006), s1 - s0, pp.max() - pp.min() + 0.012, fill=False, edgecolor=RED, lw=0.7))
a.plot([], [], color=RED, lw=0.7, label="번스타인 범위 상자")
a.set_xlim(0, hh); a.set_xlabel("한 걸음 안의 시간 s"); a.set_ylabel("값"); a.set_xticks([0, hh / 2, hh]); a.set_xticklabels(["0", "h/2", "h"])
a.legend(loc="lower right", fontsize=6.3, frameon=False)
tt = a.text(0, 1.03, "(나) 증명서의 한 걸음 (q 개 하위 걸음)", transform=a.transAxes, fontsize=8.5, fontweight="bold", va="bottom"); tt._ide_key = True
check_clip(fig, "fig_scheme_ops"); fig.savefig("fig_scheme_ops.png", dpi=220); plt.close(fig)

# ---------------------------------------------------------------- figure 2: orders of discretization
fig, ax = plt.subplots(1, 3, figsize=(7.2, 2.6), gridspec_kw=dict(wspace=0.12, left=0.04, right=0.99, bottom=0.12, top=0.84))
titles = ["공간 먼저 (선 방법)", "시간 먼저 (로테 방법)", "시공간 동시 (dG)"]
for a, ttl in zip(ax, titles):
    a.set_xlim(0, 1); a.set_ylim(0, 1); a.set_xticks([]); a.set_yticks([]); a.set_xlabel("x", fontsize=7); a.set_ylabel("t", fontsize=7)
    tt = a.text(0, 1.03, ttl, transform=a.transAxes, fontsize=8, fontweight="bold", va="bottom"); tt._ide_key = True
a = ax[0]
for xi in np.linspace(0.05, 0.95, 10): a.plot([xi, xi], [0, 1], color=BLUE, lw=0.9)
for t in np.linspace(0, 1, 9): a.plot([0, 1], [t, t], color=GRAY, lw=0.5, ls=":")
a.text(0.5, 0.5, "고정 격자 위의\n반이산계 → 시간 후보", fontsize=6.5, ha="center", va="center", bbox=dict(fc="white", ec="none", alpha=0.85))
a = ax[1]; ts = np.linspace(0, 1, 9)
for k in range(8):
    c = 0.15 + 0.7 * ts[k]; xf_ = np.linspace(0, 1, 2001); om = 1 + 6 * np.exp(-((xf_ - c) / 0.08) ** 2)
    cum = np.concatenate([[0], np.cumsum(0.5 * (om[1:] + om[:-1]) * np.diff(xf_))]); cum /= cum[-1]
    xk = np.interp(np.linspace(0, 1, 14), cum, xf_)
    for xi in xk: a.plot([xi, xi], [ts[k], ts[k + 1]], color=ORANGE, lw=0.8)
    a.plot([0, 1], [ts[k], ts[k]], color=GRAY, lw=0.6)
a.text(0.5, 0.94, "걸음마다 타원형 문제, 격자 재배치", fontsize=6.3, ha="center", va="top", bbox=dict(fc="white", ec="none", alpha=0.85))
a = ax[2]; ts = np.linspace(0, 1, 5)
```

```
for k in range(4):
    a.add_patch(plt.Rectangle((0.0, ts[k] + 0.012), 1.0, ts[k + 1] - ts[k] - 0.024, facecolor="#e8f4ee",
 edgecolor=AQUA, lw=1))
    for xi in np.linspace(0.05, 0.95, 10): a.plot([xi, xi], [ts[k] + 0.012, ts[k + 1] - 0.012], color=AQ
UA, lw=0.5)
a.text(0.5, 0.5, "시간 조각 안은 다항식,\n조각 사이는 도약", fontsize=6.5, ha="center", va="center", bbo
x=dict(fc="white", ec="none", alpha=0.85))
check_clip(fig, "fig_scheme_orders"); fig.savefig("fig_scheme_orders.png", dpi=220); plt.close(fig)
print("ok")
```

Verifier/Core.lean — 수학적 핵심의 Lean 4 형식 증명(14.2절, Mathlib)

```
import Verifier.Imports

/-!
# 시간 스킴 검증기 — 수학적 핵심의 형식 증명 (Lean 4 + Mathlib)

이 파일은 검증기가 판정에 쓰는 명제 가운데 순수 수학인 부분을 기계로 검증한다.
파이썬 코드가 이 명제를 바르게 옮겼는지는 이 파일의 범위 밖이다.
-/

open Finset Real

namespace Verifier

/-! ## 명제 5.1–5.2: 나머지 사다리 `ε_n = 1/(n!·n)` -/

/-- 핵심 부등식 `(n+2)/((n+1)!(n+1)) ≤ 1/(n!·n)`, 곧 `n(n+2) ≤ (n+1)^2`. -/
theorem ladder_key (n : ℕ) (hn : 0 < n) :
    ((n : ℝ) + 2) / (((n + 1).factorial : ℝ) * ((n : ℝ) + 1)) ≤ 1 / ((n.factorial : ℝ) * n) := by
  have hF : (0 : ℝ) < n.factorial := by exact_mod_cast Nat.factorial_pos n
  have hn' : (0 : ℝ) < n := by exact_mod_cast hn
  rw [Nat.factorial_succ, Nat.cast_mul, div_le_div_iff₀ (by positivity) (by positivity)]
  push_cast
  nlinarith [hF, hn', mul_pos hF hn']

/-- 명제 5.1 (위쪽): `e - S_n ≤ 1/(n!·n)`. -/
theorem ladder_upper (n : ℕ) (hn : 0 < n) :
    Real.exp 1 - ∑ m ∈ range (n + 1), (1 : ℝ) ^ m / m.factorial ≤ 1 / ((n.factorial : ℝ) * n) := by
  have h := Real.exp_bound (x := 1) (by norm_num) (Nat.succ_pos n)
  have h1 := (le_abs_self _).trans h
  refine h1.trans (le_of_eq_of_le ?_ (ladder_key n hn))
  simp only [abs_one, one_pow, one_mul, Nat.succ_eq_add_one]
  push_cast; ring

/-- 명제 5.1 (아래쪽): `0 < e - S_n`. -/
theorem ladder_pos (n : ℕ) :
    0 < Real.exp 1 - ∑ m ∈ range (n + 1), (1 : ℝ) ^ m / m.factorial := by
  have h := Real.sum_le_exp_of_nonneg (x := 1) (by norm_num) (n + 2)
  rw [sum_range_succ] at h
  have hp : (0 : ℝ) < (1 : ℝ) ^ (n + 1) / ((n + 1).factorial : ℝ) := by positivity
  linarith

/-- 명제 5.2: `0 ≤ θ ≤ 1`이면 한 걸음 나머지 상한 `τ(θ,n) ≤ ε_n`. -/
theorem tau_le_ladder (θ : ℝ) (n : ℕ) (hn : 0 < n) (h0 : 0 ≤ θ) (h1 : θ ≤ 1) :
    θ ^ (n + 1) / ((n + 1).factorial : ℝ) * (((n : ℝ) + 2) / ((n : ℝ) + 2 - θ))
      ≤ 1 / ((n.factorial : ℝ) * n) := by
  have hF : (0 : ℝ) < (n + 1).factorial := by exact_mod_cast Nat.factorial_pos (n + 1)
  have hpow : θ ^ (n + 1) ≤ 1 := pow_le_one₀ h0 h1
  have hden : (n : ℝ) + 1 ≤ (n : ℝ) + 2 - θ := by linarith
  have hfrac : ((n : ℝ) + 2) / ((n : ℝ) + 2 - θ) ≤ ((n : ℝ) + 2) / ((n : ℝ) + 1) :=
    div_le_div_of_nonneg_left (by positivity) (by positivity) hden
  calc θ ^ (n + 1) / ((n + 1).factorial : ℝ) * (((n : ℝ) + 2) / ((n : ℝ) + 2 - θ))
      ≤ 1 / ((n + 1).factorial : ℝ) * (((n : ℝ) + 2) / ((n : ℝ) + 1)) := by
        apply mul_le_mul (div_le_div_of_nonneg_right hpow hF.le) hfrac (by
          have : (0 : ℝ) < (n : ℝ) + 2 - θ := by linarith
          positivity) (by positivity)
    _ = ((n : ℝ) + 2) / (((n + 1).factorial : ℝ) * ((n : ℝ) + 1)) := by
        field_simp
    _ ≤ 1 / ((n.factorial : ℝ) * n) := ladder_key n hn

/-! ## 명제 5.3: 망원 합 (노름 환에서) -/

/-- `‖S^(N+1) - E^(N+1)‖ ≤ (N+1)·δ·a^N` (`‖S‖, ‖E‖ ≤ a`, `‖S - E‖ ≤ δ`). -/
theorem telescoping {R : Type*} [NormedRing R] (S E : R) (a δ : ℝ)
    (hS : ‖S‖ ≤ a) (hE : ‖E‖ ≤ a) (hd : ‖S - E‖ ≤ δ) (N : ℕ) :
    ‖S ^ (N + 1) - E ^ (N + 1)‖ ≤ (N + 1) * δ * a ^ N := by
  have ha : 0 ≤ a := (norm_nonneg _).trans hS
  have hδ : 0 ≤ δ := (norm_nonneg _).trans hd
```

```
  induction N with
  | zero => simpa using hd
  | succ N ih =>
    have key : S ^ (N + 2) - E ^ (N + 2) = S * (S ^ (N + 1) - E ^ (N + 1)) + (S - E) * E ^ (N + 1) := by
      rw [pow_succ' S (N + 1), pow_succ' E (N + 1)]; simp only [mul_sub, sub_mul]; abel
    rw [key]
    calc ‖S * (S ^ (N + 1) - E ^ (N + 1)) + (S - E) * E ^ (N + 1)‖
        ≤ ‖S‖ * ‖S ^ (N + 1) - E ^ (N + 1)‖ + ‖S - E‖ * ‖E ^ (N + 1)‖ :=
          (norm_add_le _ _).trans (add_le_add (norm_mul_le _ _) (norm_mul_le _ _))
      _ ≤ a * ((N + 1) * δ * a ^ N) + δ * a ^ (N + 1) := by
          apply add_le_add
          · exact mul_le_mul hS ih (norm_nonneg _) ha
          · exact mul_le_mul hd ((norm_pow_le' E (Nat.succ_pos N)).trans
              (pow_le_pow_left₀ (norm_nonneg _) hE _)) (norm_nonneg _) hδ
      _ = ((N + 1 : ℕ) + 1) * δ * a ^ (N + 1) := by push_cast; ring

/-- `(1 + τ)^N ≤ e^(Nτ)`: 명제 5.3의 마지막 꼴. -/
theorem one_add_pow_le_exp (τ : ℝ) (hτ : 0 ≤ τ) (N : ℕ) : (1 + τ) ^ N ≤ Real.exp (N * τ) := by
  calc (1 + τ) ^ N ≤ (Real.exp τ) ^ N := by
        apply pow_le_pow_left₀ (by linarith) ; linarith [Real.add_one_le_exp τ]
    _ = Real.exp (N * τ) := by rw [← Real.exp_nat_mul]

/-! ## 표준 부동소수 모형: `|∏(1+δ_i) - 1| ≤ γ_n = n·u/(1 - n·u)` -/

theorem gamma_bound (u : ℝ) (n : ℕ) (δ : Fin n → ℝ) (hu : 0 ≤ u) (hδ : ∀ i, |δ i| ≤ u)
    (hnu : (n : ℝ) * u < 1) :
    |∏ i, (1 + δ i) - 1| ≤ (n : ℝ) * u / (1 - n * u) := by
  rcases Nat.eq_zero_or_pos n with rfl | hn
  · simp
  have hu1 : u ≤ 1 := by
    have : (1 : ℝ) ≤ n := by exact_mod_cast hn
    nlinarith
  have hpos : 0 < 1 - (n : ℝ) * u := by linarith
  have hup : ∏ i, (1 + δ i) ≤ (1 + u) ^ n := by
    calc ∏ i, (1 + δ i) ≤ ∏ _i : Fin n, (1 + u) := by
          apply Finset.prod_le_prod₀
          · intro i _; linarith [(abs_le.mp (hδ i)).1]
          · intro i _; linarith [(abs_le.mp (hδ i)).2]
      _ = (1 + u) ^ n := by simp
  have hlo : (1 - u) ^ n ≤ ∏ i, (1 + δ i) := by
    calc (1 - u) ^ n = ∏ _i : Fin n, (1 - u) := by simp
      _ ≤ ∏ i, (1 + δ i) := by
          apply Finset.prod_le_prod₀
          · intro i _; linarith
          · intro i _; linarith [(abs_le.mp (hδ i)).1]
  have hexp : (1 + u) ^ n ≤ 1 / (1 - n * u) := by
    calc (1 + u) ^ n ≤ Real.exp (n * u) := one_add_pow_le_exp u hu n
      _ ≤ 1 / (1 - n * u) := by
        rcases eq_or_lt_of_le (mul_nonneg (Nat.cast_nonneg n) hu) with h0 | h0
        · rw [← h0]; simp
        · exact (Real.exp_bound_div_one_sub_of_interval' h0 hnu).le
  have hbern : 1 - (n : ℝ) * u ≤ (1 - u) ^ n := by
    have h := one_add_mul_le_pow (a := -u) (by linarith) n
    rw [← sub_eq_add_neg, mul_neg, ← sub_eq_add_neg] at h
    exact h
  have hg : (n : ℝ) * u ≤ (n : ℝ) * u / (1 - n * u) := by
    rw [le_div_iff₀ hpos]; nlinarith [mul_nonneg (Nat.cast_nonneg n) hu]
  rw [abs_le]
  constructor
  · linarith
  · have : 1 / (1 - (n : ℝ) * u) - 1 = (n : ℝ) * u / (1 - n * u) := by field_simp; ring
    linarith

/-! ## 명제 1.4: 길이와 각도 (명제 6.1의 벡터 꼴) -/

theorem length_angle_split {V : Type*} [NormedAddCommGroup V] [InnerProductSpace ℝ V] (y u : V) :
    ‖y - u‖ ^ 2 = (‖y‖ - ‖u‖) ^ 2 + 4 * ‖y‖ * ‖u‖ * Real.sin (InnerProductGeometry.angle y u / 2) ^ 2 :=
 by
  have h1 := norm_sub_sq_real y u
  have h2 := InnerProductGeometry.cos_angle_mul_norm_mul_norm y u
  have h3 : Real.sin (InnerProductGeometry.angle y u / 2) ^ 2 = (1 - Real.cos (InnerProductGeometry.angl
e y u)) / 2 := by
    have hc := Real.cos_two_mul (InnerProductGeometry.angle y u / 2)
    rw [show 2 * (InnerProductGeometry.angle y u / 2) = InnerProductGeometry.angle y u by ring, Real.cos
_sq'] at hc
    linarith
  rw [h3, h1, ← h2]; ring

/-! ## 명제 6.2의 계수 계산 (mod 4 규칙) -/

```

```
/-- 홀수 차수 `n = 2m+1`: `i^(n+1) = (-1)^(m+1)`. -/
theorem modfour_odd (m : ℕ) : Complex.I ^ (2 * m + 2) = (-1 : ℂ) ^ (m + 1) := by
  rw [show 2 * m + 2 = 2 * (m + 1) by ring, pow_mul, Complex.I_sq]

/-- 짝수 차수 `n = 2m`: 첫 항의 계수 `i^(n+2)(1/(n+2)! - 1/(n+1)!) = -(-1)^(m+1)(n+1)/(n+2)!`. -/
theorem modfour_even (m : ℕ) :
    Complex.I ^ (2 * m + 2) * ((1 : ℂ) / (2 * m + 2).factorial - 1 / (2 * m + 1).factorial)
      = -((-1 : ℂ) ^ (m + 1)) * (((2 * m + 1 : ℕ) : ℂ) / (2 * m + 2).factorial) := by
  rw [modfour_odd]
  have hf : ((2 * m + 2).factorial : ℂ) = (2 * m + 2 : ℂ) * (2 * m + 1).factorial := by
    rw [show 2 * m + 2 = (2 * m + 1) + 1 by ring, Nat.factorial_succ]; push_cast; ring
  have h1 : ((2 * m + 1).factorial : ℂ) ≠ 0 := by exact_mod_cast (Nat.factorial_pos _).ne'
  have h2 : (2 * m + 2 : ℂ) ≠ 0 := by exact_mod_cast (show 2 * m + 2 ≠ 0 by omega)
  rw [hf]; field_simp; push_cast; ring

/-! ## 13절: 국소화가 정확한 몫인 이유 (점화식의 닫힌 꼴) -/

/-- `E(k+1) = g k * (E k + c k)`이면 `E N = (∏ g) E 0 + Σ_k (∏_{k ≤ j < N} g j) c k`. -/
theorem localization_closed (g c E : ℕ → ℝ) (hE : ∀ k, E (k + 1) = g k * (E k + c k)) (N : ℕ) :
    E N = (∏ k ∈ range N, g k) * E 0 + Σ k ∈ range N, (∏ j ∈ Ico k N, g j) * c k := by
  induction N with
  | zero => simp
  | succ N ih =>
    have hs : Σ k ∈ range N, (∏ j ∈ Ico k (N + 1), g j) * c k
        = g N * Σ k ∈ range N, (∏ j ∈ Ico k N, g j) * c k := by
      rw [mul_sum]; apply sum_congr rfl; intro k hk
      rw [prod_Ico_succ_top (mem_range.mp hk).le]; ring
    rw [hE, ih, prod_range_succ, sum_range_succ, hs, prod_Ico_succ_top le_rfl, Ico_self, prod_empty]
    ring

/-! ## 명제 1.3과 오차 전파: `a(n+1) ≤ ρ n * a n + b n` 의 닫힌 상한 -/

theorem propagation (a ρ b : ℕ → ℝ) (hρ : ∀ n, 0 ≤ ρ n) (hrec : ∀ n, a (n + 1) ≤ ρ n * a n + b n) (N : ℕ
) :
    a N ≤ (∏ k ∈ range N, ρ k) * a 0 + Σ k ∈ range N, (∏ j ∈ Ico (k + 1) N, ρ j) * b k := by
  induction N with
  | zero => simp
  | succ N ih =>
    have hs : Σ k ∈ range N, (∏ j ∈ Ico (k + 1) (N + 1), ρ j) * b k
        = ρ N * Σ k ∈ range N, (∏ j ∈ Ico (k + 1) N, ρ j) * b k := by
      rw [mul_sum]; apply sum_congr rfl; intro k hk
      rw [prod_Ico_succ_top (mem_range.mp hk)]; ring
    rw [prod_range_succ, sum_range_succ, hs, Ico_self, prod_empty]
    calc a (N + 1) ≤ ρ N * a N + b N := hrec N
      _ ≤ ρ N * ((∏ k ∈ range N, ρ k) * a 0 + Σ k ∈ range N, (∏ j ∈ Ico (k + 1) N, ρ j) * b k) + b N :=
by
          gcongr; exact hρ N
      _ = _ := by ring

/-- 명제 1.3의 반례 논리: 정확한 반복 `w`와 계산된 반복 `x`의 차이가 `e` 이하이고
`‖x‖ - e`가 한계 `L`을 넘으면 `‖w‖ > L` (정확한 흐름에 없는 증폭). -/
theorem amplification_witness {V : Type*} [NormedAddCommGroup V] (w x : V) (e L : ℝ)
    (he : ‖w - x‖ ≤ e) (hx : L < ‖x‖ - e) : L < ‖w‖ := by
  have := norm_sub_norm_le x w
  rw [norm_sub_rev] at he
  linarith

/-- 걸음 사상의 곱 `P (n+1) = Φ n * P n`, `P 0 = 1`. -/
def stepProd {R : Type*} [Ring R] (Φ : ℕ → R) : ℕ → R
  | 0 => 1
  | n + 1 => Φ n * stepProd Φ n

/-- 일단계 사상의 노름 곱으로 증폭 없음: `‖Φ_{N-1} ⋯ Φ_0‖ ≤ ∏ ‖Φ_k‖`. -/
theorem product_norm_le {R : Type*} [NormedRing R] [NormOneClass R] (Φ : ℕ → R) (N : ℕ) :
    ‖stepProd Φ N‖ ≤ ∏ i ∈ range N, ‖Φ i‖ := by
  induction N with
  | zero => simp [stepProd]
  | succ N ih =>
    rw [stepProd, prod_range_succ]
    calc ‖Φ N * stepProd Φ N‖ ≤ ‖Φ N‖ * ‖stepProd Φ N‖ := norm_mul_le _ _
      _ ≤ ‖Φ N‖ * ∏ i ∈ range N, ‖Φ i‖ := by gcongr
      _ = _ := by ring

/-- 암시적 풀이의 역행렬 한계(명제 1.3): 계산한 근사 역 `Z`에서 `‖1 - M Z‖ < 1`이면
`M`의 오른쪽 역 `W`가 있고 `‖W‖ ≤ ‖Z‖ / (1 - ‖1 - M Z‖)`. -/
theorem inverse_norm_bound {R : Type*} [NormedRing R] [NormOneClass R] [HasSummableGeomSeries R]
    (M Z : R) (h : ‖1 - M * Z‖ < 1) :
    M * (Z * Σ' n : ℕ, (1 - M * Z) ^ n) = 1 ∧
      ‖Z * Σ' n : ℕ, (1 - M * Z) ^ n‖ ≤ ‖Z‖ / (1 - ‖1 - M * Z‖) := by
  have hpos : 0 < 1 - ‖1 - M * Z‖ := by linarith
```

```
  constructor
  · have := mul_neg_geom_series (1 - M * Z) h
    rw [sub_sub_cancel] at this
    rw [← mul_assoc]; exact this
  · have hg := tsum_geometric_le_of_norm_lt_one (1 - M * Z) h
    rw [norm_one, sub_self, zero_add] at hg
    calc ‖Z * Σ' n : ℕ, (1 - M * Z) ^ n‖ ≤ ‖Z‖ * ‖Σ' n : ℕ, (1 - M * Z) ^ n‖ := norm_mul_le _ _
      _ ≤ ‖Z‖ * (1 - ‖1 - M * Z‖)⁻¹ := by gcongr
      _ = ‖Z‖ / (1 - ‖1 - M * Z‖) := by rw [div_eq_mul_inv]

end Verifier
```

작동 확인 run.py — A1: 경로 A를 1–4차원 텐서 격자에서 판정

```
"""Use the existing numide code unchanged on 1D/2D/3D semi-discrete systems (tensor-product FDM, built here by kron)."""
import sys, time, json, numpy as np
sys.path.insert(0, "../numide")
from operators import fdm_vertex, fvm_cell
from semidisc import Sys, cand_rk4h, true_error
from cert import certify_best
dim, n, kind = int(sys.argv[1]), int(sys.argv[2]), sys.argv[3]
L1, w1, x1 = (fdm_vertex if kind == "fdm" else fvm_cell)(n, 1.0)
L, w, I1 = L1, w1, np.eye(n)
for _ in range(dim - 1):
    L = np.kron(L, I1) + np.kron(np.eye(L.shape[0]), L1); w = np.kron(w, w1)
N = L.shape[0]
grids = np.meshgrid(*([x1] * dim), indexing="ij"); r2 = sum((g - 0.5) ** 2 for g in grids).ravel()
u0 = np.concatenate([0.5 + 0.3 * np.exp(-8 * r2), 0.2 + 0.0 * r2])
spec = dict(m=2, N=N, L=L, w=w, D=[0.02, 0.01], T=1.0, u0=u0,
            R=lambda s: [s[0] * (1 - s[0]) - s[0] * s[1], 0.5 * s[0] * s[1] - 0.3 * s[1]])
S = Sys(spec); C = cand_rk4h(S, int(sys.argv[4]) if len(sys.argv) > 4 else 40)
t0 = time.time(); hist, best = certify_best(S, C); wall = time.time() - t0
print(json.dumps(dict(dim=dim, n=n, kind=kind, unknowns=2 * N, proved=best["proved_full"], t_star=best["t_star"],
                      bound=best["eps"], true_err=true_error(S, C), wall_s=round(wall, 1)), default=float))
```

작동 확인 schemes.py — A2: 여러 시간 스킴의 출력을 경로 A로 판정

```
"""many time schemes, one verifier: path A on a 2D two-species reaction-diffusion system (tensor FDM 6x6)"""
import sys, time, json, numpy as np
sys.path.insert(0, "../numide")
from scipy.integrate import solve_ivp
from scipy.linalg import expm
from operators import fdm_vertex
from semidisc import Sys, cand_from_values, true_error
from cert import certify_best
n1 = 6; L1, w1, x1 = fdm_vertex(n1, 1.0); I1 = np.eye(n1)
L = np.kron(L1, I1) + np.kron(I1, L1); w = np.kron(w1, w1); N = L.shape[0]
g = np.meshgrid(x1, x1, indexing="ij"); r2 = ((g[0] - .5) ** 2 + (g[1] - .5) ** 2).ravel()
u0 = np.concatenate([0.5 + 0.3 * np.exp(-8 * r2), 0.2 + 0 * r2]); D = [0.02, 0.01]
Rf = lambda s: [s[0] * (1 - s[0]) - s[0] * s[1], 0.5 * s[0] * s[1] - 0.3 * s[1]]
S = Sys(dict(m=2, N=N, L=L, w=w, D=D, T=1.0, u0=u0, R=Rf)); F = S.F; T = 1.0
Ad = np.kron(np.diag(D), L)                                  # linear (diffusion) part, for splitting / IMEX
def reac(z): u = z.reshape(2, N); r = Rf(list(u)); return np.concatenate([np.asarray(v) * np.ones(N) for v in r])

def uniform(step, n):
    h = T / n; t = np.linspace(0, T, n + 1); Y = [u0.copy()]
    for k in range(n): Y.append(step(Y[-1], h))
    return t, np.array(Y)
def rk4(y, h):
    k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h / 2 * k2); k4 = F(y + h * k3); return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
def cn(y, h):                                                # Crank-Nicolson (implicit trapezoid), Newton
    z = y.copy(); fy = F(y)
    for _ in range(20):
        G = z - y - h / 2 * (fy + F(z)); J = np.eye(len(z)) - h / 2 * S.Jnum(z.reshape(2, N)); d = np.linalg.solve(J, G); z -= d
        if np.max(np.abs(d)) < 1e-15: break
    return z
E_half = {}
def strang(y, h):                                            # Strang splitting: exact diffusion half steps, RK4 reaction
```

```
    if h not in E_half: E_half[h] = expm(h / 2 * Ad)
    E = E_half[h]; z = E @ y
    k1 = reac(z); k2 = reac(z + h / 2 * k1); k3 = reac(z + h / 2 * k2); k4 = reac(z + h * k3); z = z + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
    return E @ z
def ab3(n):                                                 # Adams-Bashforth 3 (multistep), RK4 start
    h = T / n; t = np.linspace(0, T, n + 1); Y = [u0.copy()]
    for k in range(2): Y.append(rk4(Y[-1], h))
    Fs = [F(y) for y in Y]
    for k in range(2, n):
        Y.append(Y[-1] + h / 12 * (23 * Fs[-1] - 16 * Fs[-2] + 5 * Fs[-3])); Fs.append(F(Y[-1]))
    return t, np.array(Y)
def adaptive(method, rtol):
    sol = solve_ivp(lambda t, z: F(z), (0, T), u0, method=method, rtol=rtol, atol=rtol * 1e-2,
                    jac=(lambda t, z: S.Jnum(z.reshape(2, N))) if method in ("BDF", "Radau") else None)
    return sol.t, sol.y.T

cases = {"RK4 (uniform)": lambda: uniform(rk4, 40), "Crank-Nicolson (implicit)": lambda: uniform(cn, 400),
         "Strang splitting": lambda: uniform(strang, 200), "Adams-Bashforth 3 (multistep)": lambda: ab3(400),
         "RK45 (adaptive)": lambda: adaptive("RK45", 1e-10), "DOP853 (adaptive)": lambda: adaptive("DOP853", 1e-11),
         "BDF (adaptive, implicit multistep)": lambda: adaptive("BDF", 1e-10), "Radau IIA (adaptive)": lambda: adaptive("Radau", 1e-10)}
which = sys.argv[1:] or list(cases)
for name in which:
    t, Y = cases[name](); C, hk = cand_from_values(S, t, Y); t0 = time.time()
    hist, best = certify_best(S, C, hk=hk, rounds=4)
    wh = best.get("where"); top = wh[0] if isinstance(wh, list) and wh else None
    print(json.dumps(dict(scheme=name, steps=len(hk), h_min=float(hk.min()), h_max=float(hk.max()), proved=best["proved_full"],
                          t_star=round(best["t_star"], 4), bound=best["eps"], true_err=true_error(S, C, hk)[0], repaired=best.get("repaired"),
                          top=top and dict(t=[round(v, 4) for v in top["t"]], term=top["term"], share=round(top["share"], 3)),
                          wall=round(time.time() - t0, 1)), default=float), flush=True)
```

작동 확인 nd_test.py — A3·A4: 경로 B를 임의 차원, 공간 먼저·시간 먼저로 판정하고 국소화

```
import sys, time, numpy as np
sys.path.insert(0, "../numide")
from pde_judge_nd import JudgeND, localize
from orders_nd import mol_nd, rothe_nd, kron_operator
d, nx, nt, mode = int(sys.argv[1]), int(sys.argv[2]), int(sys.argv[3]), sys.argv[4]
amp = float(sys.argv[5]) if len(sys.argv) > 5 else 0.0
R = lambda s: [s[0] - s[0] ** 2]
def f0(X):
    p = 1.0
    for x in X: p = p * np.cos(np.pi * x)
    return 0.5 + 0.3 * p
def u0box(a, xl, xh):                       # cos(pi x) is decreasing on [0, 1]: interval product of [cos(pi xh), cos(pi xl)]
    lo = np.ones_like(xl[0]); hi = np.ones_like(xl[0])
    for l, h_ in zip(xl, xh):
        cl, ch = np.cos(np.pi * np.clip(h_, 0, 1)), np.cos(np.pi * np.clip(l, 0, 1))
        c = np.stack([lo * cl, lo * ch, hi * cl, hi * ch]); lo, hi = c.min(0), c.max(0)
    return 0.5 + 0.3 * lo - 1e-14, 0.5 + 0.3 * hi + 1e-14
def u0grad(a, xl, xh):                # d/dx_i f0 = -0.3 pi sin(pi x_i) prod_{j != i} cos(pi x_j): interval products
    d = len(xl); cosI = []; sinI = []
    for l, h_ in zip(xl, xh):
        l = np.clip(l, 0, 1); h_ = np.clip(h_, 0, 1)
        cosI.append((np.cos(np.pi * h_) - 1e-15, np.cos(np.pi * l) + 1e-15))
        sl, sh = np.sin(np.pi * l), np.sin(np.pi * h_); smax = np.where((l <= 0.5) & (h_ >= 0.5), 1.0, np.maximum(sl, sh))
        sinI.append((np.minimum(sl, sh) - 1e-15, smax + 1e-15))
    out = []
    for i in range(d):
        lo, hi = sinI[i]
        for j in range(d):
            if j == i: continue
            c = np.stack([lo * cosI[j][0], lo * cosI[j][1], hi * cosI[j][0], hi * cosI[j][1]]); lo, hi = c.min(0), c.max(0)
        k = -0.3 * np.pi; out.append((k * hi - 1e-14, k * lo + 1e-14))
    return out
prob = dict(m=1, D=[0.05], R=R, T=0.5, u0=[f0])
xs = [np.linspace(0, 1, nx)] * d
```

```
t0 = time.time()
pieces = mol_nd(prob, xs, nt) if mode == "mol" else rothe_nd(prob, xs, nt, remesh=max(2, nt // 4))[0]
tc = time.time() - t0
if amp:                                  # plant an error in one piece at one node
    k = nt // 2; idx = tuple([nx // 3] * d) + (1,); pieces[k].V[0][idx] += amp
J = JudgeND(dict(m=1, D=[0.05], R=R, T=0.5, box=([0.0], [1.0]), dim=d))
op = lambda xs_: kron_operator(xs_)
res = J.run(pieces, u0box, op=op, u0grad=u0grad); rows, tot = localize(res, top=4)
print(f"d={d} nx={nx} nt={nt} {mode} unknowns={nx**d} bound={res['bound']:.3e} e0={res['e0']:.2e} resid=
{res['residual']:.2e} jumps={res['jumps']:.2e} mu1max={res['mu1_max']:.2e} cand={tc:.1f}s judge={res['wa
ll']:.1f}s")
print("  shares", {k: round(v, 4) for k, v in tot.items()}, "sum", round(sum(tot.values()), 8))
if amp: print("  planted t=[%.3f,%.3f] node x=%.3f" % (pieces[nt//2].t0, pieces[nt//2].t0 + pieces[nt//2
].h, pieces[nt//2].xs[0][nx // 3]))
for r in rows: print("  ", r["cause"], "t=[%.3f,%.3f]" % tuple(r["t"]), r["x"] and ["[%.3f,%.3f]" % tupl
e(v) for v in r["x"]], "%.3f" % r["share"])
```

작동 확인 ref_check.py — A3: 촘촘한 기준 해와 비교한 실제 L^1 오차(판정에 쓰지 않는 비교)

```
import sys, numpy as np
sys.path.insert(0, "../numide")
from orders_nd import mol_nd, rothe_nd
from pde_judge_nd import tensor_spline, eval_tensor
d, nx, nt, nref, ntref, mode = int(sys.argv[1]), int(sys.argv[2]), int(sys.argv[3]), int(sys.argv[4]), i
nt(sys.argv[5]), sys.argv[6]
def f0(X):
    p = 1.0
    for x in X: p = p * np.cos(np.pi * x)
    return 0.5 + 0.3 * p
prob = dict(m=1, D=[0.05], R=lambda s: [s[0] - s[0] ** 2], T=0.5, u0=[f0])
def end_vals(pcs, grid):
    p = pcs[-1]; K = p.V[0].shape[-1]; end = (p.V[0] * p.h ** np.arange(K)).sum(-1)
    return eval_tensor(p.xs, tensor_spline(p.xs, end[..., None])[..., 0], grid)
grid = [np.linspace(0, 1, 4 * nref + 1)[:-1] + 0.5 / (4 * nref)] * d     # midpoint rule
cand = mol_nd(prob, [np.linspace(0, 1, nx)] * d, nt) if mode == "mol" else rothe_nd(prob, [np.linspace(0
, 1, nx)] * d, nt, remesh=max(2, nt // 4))[0]
ref = mol_nd(prob, [np.linspace(0, 1, nref)] * d, ntref)
print(f"d={d} {mode}: L1 error at T (midpoint quadrature vs fine reference) = {np.mean(np.abs(end_vals(c
and, grid) - end_vals(ref, grid))):.3e}")
```

작동 확인 cmp1d.py — A3: 1차원 모듈과의 일치

```
import sys, numpy as np
sys.path.insert(0, "../numide")
from orders_nd import mol_nd
from pde_judge import Judge, Piece
f0 = lambda X: 0.5 + 0.3 * np.cos(np.pi * X[0])
R = lambda s: [s[0] - s[0] ** 2]
pcs = mol_nd(dict(m=1, D=[0.05], R=R, T=0.5, u0=[f0]), [np.linspace(0, 1, 21)], 50)
p1 = [Piece(p.t0, p.h, p.xs[0], p.V) for p in pcs]
J = Judge(dict(m=1, D=[0.05], R=R, T=0.5, box=([0.0], [1.0])))
u0box = lambda a, xl, xh: (0.5 + 0.3 * np.cos(np.pi * xh) - 1e-15, 0.5 + 0.3 * np.cos(np.pi * xl) + 1e-1
5)
r = J.run(p1, u0box); print("1D module: bound %.4e e0 %.3e resid %.4e jumps %.2e" % (r["bound"], r["e0"]
, r["residual"], r["jumps"]))
```

작동 확인 pathB_bdf.py — A3: 경로 B에 적응형 스킴 출력

```
import sys, time, numpy as np
sys.path.insert(0, "../numide")
from scipy.integrate import solve_ivp
from orders_nd import pieces_from_steps, kron_operator, _RD, _init
from pde_judge_nd import JudgeND, localize
d, nx, method = int(sys.argv[1]), int(sys.argv[2]), sys.argv[3]
R = lambda s: [s[0] - s[0] ** 2]
def f0(X):
    p = 1.0
    for x in X: p = p * np.cos(np.pi * x)
    return 0.5 + 0.3 * p
def u0box(a, xl, xh):                        # cos(pi x) is decreasing on [0, 1]: interval product of [co
s(pi xh), cos(pi xl)]
    lo = np.ones_like(xl[0]); hi = np.ones_like(xl[0])
    for l, h_ in zip(xl, xh):
        cl, ch = np.cos(np.pi * np.clip(h_, 0, 1)), np.cos(np.pi * np.clip(l, 0, 1))
        c = np.stack([lo * cl, lo * ch, hi * cl, hi * ch]); lo, hi = c.min(0), c.max(0)
    return 0.5 + 0.3 * lo - 1e-14, 0.5 + 0.3 * hi + 1e-14
def u0grad(a, xl, xh):                # d/dx_i f0 = -0.3 pi sin(pi x_i) prod_{j != i} cos(pi x_j): inte
```

```
    rval products
19      d = len(xl); cosI = []; sinI = []
20      for l, h_ in zip(xl, xh):
21          l = np.clip(l, 0, 1); h_ = np.clip(h_, 0, 1)
22          cosI.append((np.cos(np.pi * h_) - 1e-15, np.cos(np.pi * l) + 1e-15))
23          sl, sh = np.sin(np.pi * l), np.sin(np.pi * h_); smax = np.where((l <= 0.5) & (h_ >= 0.5), 1.0, n
    p.maximum(sl, sh))
24          sinI.append((np.minimum(sl, sh) - 1e-15, smax + 1e-15))
25      out = []
26      for i in range(d):
27          lo, hi = sinI[i]
28          for j in range(d):
29              if j == i: continue
30              c = np.stack([lo * cosI[j][0], lo * cosI[j][1], hi * cosI[j][0], hi * cosI[j][1]]); lo, hi =
     c.min(0), c.max(0)
31          k = -0.3 * np.pi; out.append((k * hi - 1e-14, k * lo + 1e-14))
32      return out
33  prob = dict(m=1, D=[0.05], R=R, T=0.5, u0=[f0])
34  xs = [np.linspace(0, 1, nx)] * d; L = kron_operator(xs)[0]; rd = _RD(prob); y0 = _init(prob, xs)
35  sol = solve_ivp(lambda t, z: rd.F(z, L), (0, 0.5), y0, method=method, rtol=1e-8, atol=1e-10, jac=(lambda
     t, z: rd.J(z, L)) if method in ("BDF", "Radau") else None)
36  t, Y = sol.t, sol.y.T
37  pcs = pieces_from_steps(prob, [(t[k], t[k + 1] - t[k], xs, Y[k], Y[k + 1]) for k in range(len(t) - 1)])
38  J = JudgeND(dict(m=1, D=[0.05], R=R, T=0.5, box=([0.0], [1.0]), dim=d))
39  res = J.run(pcs, u0box, op=lambda x_: kron_operator(x_), u0grad=u0grad); rows, tot = localize(res, 3)
40  print(f"path B d={d} {method}: steps={len(t)-1} h in [{np.diff(t).min():.2e},{np.diff(t).max():.2e}] bou
    nd={res['bound']:.3e} shares={ {k: round(v,4) for k,v in tot.items()} } judge={res['wall']:.1f}s")
```

작동 확인 loc_test.py — A4: 경로 A 국소화

```
 1  import sys, json, numpy as np
 2  sys.path.insert(0, "../numide")
 3  from operators import fdm_vertex
 4  from semidisc import Sys, cand_rk4h
 5  from cert import certify
 6  L, w, x = fdm_vertex(21, 1.0)
 7  u0 = np.concatenate([0.5 + 0.3 * np.exp(-30 * (x - 0.3) ** 2), 0.2 + 0 * x])
 8  S = Sys(dict(m=2, N=21, L=L, w=w, D=[0.02, 0.01], T=2.0, u0=u0, R=lambda s: [s[0] * (1 - s[0]) - s[0] *
    s[1], 0.5 * s[0] * s[1] - 0.3 * s[1]]))
 9  C = cand_rk4h(S, 40)
10  # plant an error: perturb one step's candidate at component 0, node 6 (x=0.3)
11  C[25] = C[25].copy(); C[25][1, 6] += float(sys.argv[1])
12  hist, best = certify(S, C, rounds=0)
13  for r in hist: print(r["where"] if isinstance(r["where"], str) else ""); print(r["proved_full"], r["t_st
    ar"], r.get("dominant"), r.get("action")); [print(" ", w_) for w_ in r["where"][:5]]
```

작동 확인 loc_pde.py — A4: 경로 B 1차원 국소화(dG, 시간 먼저)

```
 1  import sys, time, numpy as np
 2  sys.path.insert(0, "../numide")
 3  from operators import fem_p1
 4  from orders import rothe, stdg
 5  from pde_judge import Judge, localize
 6  mode, amp = sys.argv[1], float(sys.argv[2])
 7  f0 = lambda x: 0.5 + 0.3 * np.cos(np.pi * x)
 8  prob = dict(m=1, D=[0.05], R=lambda s: [s[0] - s[0] ** 2], T=0.5, u0=[f0],
 9              op=lambda x: (*fem_p1(x)[:2], np.zeros(len(x))), robin=None)
10  pieces, meshes = (rothe(prob, N=41, hmax=0.01, remesh=10) if mode == "rothe" else stdg(prob, N=41, q=2,
    n=50))
11  if amp:                                              # plant an error: piece 20, node 12 (x ~ 0.3),
    first-order time coefficient
12      k = 20; pieces[k].V[0][12, 1] += amp
13  J = Judge(dict(m=1, D=[0.05], R=prob["R"], T=0.5, box=([0.0], [1.0])))
14  u0box = lambda a, xl, xh: (np.minimum(f0(xl), f0(xh)) - 1e-15, np.maximum(f0(xl), f0(xh)) + 1e-15)
15  r = J.run(pieces, u0box, op=prob["op"]); rows, tot = localize(r, top=5)
16  print(mode, "pieces", r["pieces"], "bound %.3e" % r["bound"], "shares", {k: round(v, 4) for k, v in tot.
    items()}, "sum", round(sum(tot.values()), 6))
17  if amp: print("planted t=[%.4f,%.4f] x~%.3f" % (pieces[20].t0, pieces[20].t0 + pieces[20].h, pieces[20].
    x[12]))
18  for w in rows: print("  t=[%.4f,%.4f]" % tuple(w["t"]), "x=" + ("[%.3f,%.3f]" % tuple(w["x"]) if w["x"]
    else "-"), w["cause"], "%.3f" % w["share"])
```

작동 확인 schemes_lib.py — A5: 시험 계와 스킴(일단계·다단계·암시적·적응형·가변 계수, 각각 틀린 판도 포함)

```
"""schemes_lib: the test system and the schemes used to check verify() — one-step, multistep, implicit, adaptive and
variable-coefficient solvers, each also in a deliberately wrong version"""
import sys, time, numpy as np
sys.path.insert(0, "../numide")
from scipy.integrate import solve_ivp
from scipy.integrate._ivp.rk import RK45, RK23
from operators import fdm_vertex
from semidisc import Sys
L, w, x = fdm_vertex(21, 1.0)
u0 = np.concatenate([0.5 + 0.3 * np.exp(-30 * (x - 0.3) ** 2), 0.2 + 0 * x])
S = Sys(dict(m=2, N=21, L=L, w=w, D=[0.02, 0.01], T=2.0, u0=u0, R=lambda s: [s[0] * (1 - s[0]) - s[0] * s[1], 0.5 * s[0] * s[1] - 0.3 * s[1]]))
def rk4(F, y, h):
    k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h / 2 * k2); k4 = F(y + h * k3); return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
def ab3_solver(N, c=(23 / 12, -16 / 12, 5 / 12)):
    def solve(F, y0, T):
        h = T / N; Y = [y0]
        for _ in range(2): Y.append(rk4(F, Y[-1], h))
        Fs = [F(y) for y in Y]
        for _ in range(2, N): Y.append(Y[-1] + h * (c[0] * Fs[-1] + c[1] * Fs[-2] + c[2] * Fs[-3])); Fs.append(F(Y[-1]))
        return np.linspace(0, T, N + 1), np.array(Y)
    return solve
def bdf2_solver(N):
    def solve(F, y0, T):
        h = T / N; Y = [y0, rk4(F, y0, h)]; d = len(y0)
        for _ in range(1, N):
            z = Y[-1].copy(); rhs = 4 / 3 * Y[-1] - 1 / 3 * Y[-2]
            for _ in range(30):
                G = z - rhs - 2 / 3 * h * F(z); e = 1e-7
                Jn = np.array([(F(z + e * np.eye(d)[i]) - F(z - e * np.eye(d)[i])) / (2 * e) for i in range(d)]).T
                dz = np.linalg.solve(np.eye(d) - 2 / 3 * h * Jn, G); z -= dz
                if np.max(np.abs(dz)) < 1e-15 * max(1, np.max(np.abs(z))): break
            Y.append(z)
        return np.linspace(0, T, N + 1), np.array(Y)
    return solve
def scipy_solver(method, rtol):
    def solve(F, y0, T):
        s = solve_ivp(lambda t, z: F(z), (0, T), y0, method=method, rtol=rtol, atol=rtol * 1e-2); return s.t, s.y.T
    return solve
def bs23_adaptive(bug=False, tol=1e-6):            # own adaptive Bogacki-Shampine 3(2); bug: wrong weight on k2
    b = np.array([2 / 9, 1 / 3 + (0.02 if bug else 0.0), 4 / 9]); bh = np.array([7 / 24, 1 / 4, 1 / 3, 1 / 8])
    def solve(F, y0, T):
        t, y, h, ts, Ys = 0.0, y0.copy(), 1e-3, [0.0], [y0.copy()]
        while t < T - 1e-14:
            h = min(h, T - t); k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + 3 * h / 4 * k2)
            yn = y + h * (b[0] * k1 + b[1] * k2 + b[2] * k3); k4 = F(yn)
            e = h * np.max(np.abs((b[0] - bh[0]) * k1 + (b[1] - bh[1]) * k2 + (b[2] - bh[2]) * k3 - bh[3] * k4))
            if e <= tol: t += h; y = yn; ts.append(t); Ys.append(y.copy())
            h *= min(4.0, max(0.2, 0.9 * (tol / max(e, 1e-300)) ** (1 / 3)))
        return np.array(ts), np.array(Ys)
    return solve

from verify import _dweights as _lagrange_dweights
def vsbdf_solver(bug=False, tol=1e-6, qmax=3, tol_res=1e-12):
    """own adaptive variable-step BDF (orders 1..qmax); bug: uses the CONSTANT-step BDF weights even when steps vary"""
    const = {1: np.array([-1.0, 1.0]), 2: np.array([0.5, -2.0, 1.5]), 3: np.array([-1 / 3, 1.5, -3.0, 11 / 6])}
    def solve(F, y0, T):
        ts, Ys, h = [0.0], [y0.copy()], 1e-3; d = len(y0)
        while ts[-1] < T - 1e-14:
            h = min(h, T - ts[-1]); n = len(ts); q = min(qmax, n); tn = ts[-1] + h
            tt = np.array(ts[-q:] + [tn]); w = (const[q] / h) if bug else _lagrange_dweights(tt)
            hist = sum(w[i] * Ys[-q + i] for i in range(q))
            yp = Ys[-1] + (Ys[-1] - Ys[-2]) * h / (ts[-1] - ts[-2]) if n > 1 else Ys[-1].copy()   # linear predictor
            y = yp.copy()
            for _ in range(50):
```

```
                Gv = hist + w[-1] * y - F(y)
                if np.max(np.abs(Gv)) <= tol_res * (1 + np.max(np.abs(F(y)))): break
                e = 1e-7 * max(1.0, np.max(np.abs(y)))
                Jn = np.array([(F(y + e * np.eye(d)[i]) - F(y - e * np.eye(d)[i])) / (2 * e) for i in ra
nge(d)]).T
                y = y - np.linalg.solve(w[-1] * np.eye(d) - Jn, Gv)
            err = np.max(np.abs(y - yp)) / (q + 1)
            if err <= tol or (bug and h <= 2e-3) or h < 1e-10:
                ts.append(tn); Ys.append(y)
            h = max(h * min(2.0, max(0.3, 0.9 * (tol / max(err, 1e-300)) ** (1 / (q + 1)))), 2e-3 if bug
 else 1e-12)
        return np.array(ts), np.array(Ys)
    return solve
A45 = np.zeros((6, 6)); A45[:, :5] = RK45.A; BS_A = np.array([[0, 0, 0], [0.5, 0, 0], [0, 0.75, 0]])
cases = {"ab3": ("애덤스–배시퍼스 3", ab3_solver(400), ("lmm", [0, 0, -1, 1], [5 / 12, -16 / 12, 23 / 12
, 0])),
         "ab3bug": ("AB3 계수 오류(23/12 → 22/12)", ab3_solver(400, (22 / 12, -16 / 12, 5 / 12)), ("lmm"
, [0, 0, -1, 1], [5 / 12, -16 / 12, 23 / 12, 0])),
         "bdf2": ("BDF2(암시적 다단계)", bdf2_solver(200), ("lmm", [1 / 3, -4 / 3, 1], [0, 0, 2 / 3])),
         "rk45": ("RK45(적응형)", scipy_solver("RK45", 1e-9), ("rk", A45, RK45.B)),
         "bs23": ("보가키–샴핀 3(2)(적응형)", bs23_adaptive(False), ("rk", BS_A, [2 / 9, 1 / 3, 4 / 9])
,
         "bs23bug": ("보가키–샴핀 3(2) 가중치 오류(적응형)", bs23_adaptive(True), ("rk", BS_A, [2 / 9, 1
 / 3, 4 / 9])),
         "bdf": ("BDF(적응형 가변 계수)", scipy_solver("BDF", 1e-9), None),
         "vsbdf": ("가변 걸음 BDF(적응형, 1–3차)", vsbdf_solver(False), ("vbdf", 1e-12, 3)),
         "vsbdfbug": ("가변 걸음 BDF, 고정 걸음 계수를 쓰는 오류", vsbdf_solver(True), ("vbdf", 1e-12, 3
))}

def rk4_bug(F, y, h):
    k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h * k2); k4 = F(y + h * k3); return y + h / 6 * (k1 +
2 * k2 + 2 * k3 + k4)
def rk4_nl_bug(F, y, h):                    # identical to RK4 on LINEAR problems (the extra term vanish
es), wrong on nonlinear ones
    k1 = F(y); k2 = F(y + h / 2 * k1); a = h / 2 * k1
    k3 = F(y + h / 2 * k2) + (F(y + a) - F(y) - F(a)); k4 = F(y + h * k3); return y + h / 6 * (k1 + 2 *
k2 + 2 * k3 + k4)
```

작동 확인 verify_test.py — A5·13.5절: 비선형계에서 시간 스킴의 에러 판정(verify)

```
"""verify(): the rigorous procedure on one-step, multistep, implicit, adaptive and variable-coefficient
schemes"""
import sys, time, numpy as np
sys.path.insert(0, "../numide")
from schemes_lib import *                       # S, rk4, rk4_bug, rk4_nl_bug, ab3_solver, bdf2_solver
, scipy_solver, bs23_adaptive, vsbdf_solver
from verify import verify, library
from six_w import format_report
L = library(); RK4c, BS3c, DP5c = L["RK4"], L["보가키–샴핀 3"], L["도르만–프린스 5(RK45)"]
AB3c = L["애덤스–배시퍼스 3"]; BDF2c = ("lmm", [1 / 3, -4 / 3, 1], [0, 0, 2 / 3], 1e-12)
def stepper(fn, N): return dict(step=fn, N=N)
C = {"rk4":      ("RK4(40걸음)", stepper(rk4, 40), RK4c, "RK4"),
     "rk4bug":   ("RK4 구현 오류(k3 한 걸음)", stepper(rk4_bug, 40), RK4c, "RK4"),
     "rk4nl":    ("RK4, 비선형 항에서만 틀린 구현", stepper(rk4_nl_bug, 40), RK4c, "RK4"),
     "rk4big":   ("RK4(4걸음)", stepper(rk4, 4), RK4c, "RK4"),
     "ab3":      ("애덤스–배시퍼스 3(400걸음)", dict(solver=ab3_solver(400)), AB3c, "AB3"),
     "ab3bug":   ("AB3 계수 오류", dict(solver=ab3_solver(400, (22 / 12, -16 / 12, 5 / 12))), AB3c, "AB3
"),
     "bdf2":     ("BDF2(200걸음)", dict(solver=bdf2_solver(200)), BDF2c, "BDF2"),
     "rk45":     ("RK45(적응형, scipy)", dict(solver=scipy_solver("RK45", 1e-9)), DP5c, "도르만–프린스 5
"),
     "bs23":     ("보가키–샴핀 3(2)(적응형)", dict(solver=bs23_adaptive(False)), BS3c, "보가키–샴핀 3"),
     "bs23bug":  ("보가키–샴핀 3(2), 가중치 오류", dict(solver=bs23_adaptive(True)), BS3c, "보가키–샴핀
3"),
     "vsbdf":    ("가변 걸음 BDF 1–3차(적응형)", dict(solver=vsbdf_solver(False)), ("vbdf", 3, 1e-12), "
가변 걸음 BDF"),
     "vsbdfbug": ("가변 걸음 BDF, 고정 걸음 계수 오류", dict(solver=vsbdf_solver(True)), ("vbdf", 3, 1e-
12), "가변 걸음 BDF"),
     "bdfscipy": ("BDF(적응형, scipy)", dict(solver=scipy_solver("BDF", 1e-9)), None, None),
     "bdfloose": ("BDF(적응형, scipy, 느슨한 허용치)", dict(solver=scipy_solver("BDF", 1e-3), factory=la
mbda f: scipy_solver("BDF", 1e-3 * f)), None, None),
     "ab3big":   ("애덤스–배시퍼스 3(20걸음)", dict(solver=ab3_solver(20), factory=lambda f: ab3_solver(
int(round(20 / f)))), AB3c, "AB3"),
     "bdf2big":  ("BDF2(10걸음)", dict(solver=bdf2_solver(10), factory=lambda f: bdf2_solver(int(round(1
0 / f)))), BDF2c, "BDF2")}
for key in sys.argv[1:]:
    nm, kw, cl, cn = C[key]; t0 = time.time()
    rep = verify(S, claim=cl, claim_name=cn or "", eps=1e-5, tau=1e-12, name=nm, **kw)
    print(format_report(rep, title=f"[{nm}]  ({time.time() - t0:.1f}초)"), flush=True); print(flush=True
```

```
)
```

작동 확인 verify_linear.py — A5·13.5절: 선형계에서 시간 스킴의 에러 판정(verify)

```
"""verify() on a LINEAR system: periodic advection, 4th-order central differences (64 nodes), black-box steps"""
import sys, time, numpy as np
sys.path.insert(0, "../numide")
from operators import fdm4_periodic
from semidisc import Sys
from verify import verify, library
from six_w import format_report
n = 64; D1f = fdm4_periodic(n)[0]; A = -np.array([D1f(e) for e in np.eye(n)]).T
x = 2 * np.pi * np.arange(n) / n
S = Sys(dict(m=1, N=n, L=A, w=np.full(n, 2 * np.pi / n), D=[1.0], T=2 * np.pi, u0=np.exp(np.cos(x)), R=lambda s: [0 * s[0]]))
def rk4(F, y, h):
    k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h / 2 * k2); k4 = F(y + h * k3); return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
def rk4_bug(F, y, h):
    k1 = F(y); k2 = F(y + h / 2 * k1); k3 = F(y + h * k2); k4 = F(y + h * k3); return y + h / 6 * (k1 + 2 * k2 + 2 * k3 + k4)
def ssprk3(F, y, h):
    a = y + h * F(y); b = 0.75 * y + 0.25 * (a + h * F(a)); return y / 3 + 2 / 3 * (b + h * F(b))
rho = np.max(np.abs(np.linalg.eigvals(A))); T = 2 * np.pi; L = library()
cases = {"rk4": ("RK4, 안정 걸음", rk4, int(np.ceil(T * rho / 2.0)), L["RK4"], "RK4"),
         "rk4bug": ("RK4 구현 오류(k3)", rk4_bug, int(np.ceil(T * rho / 2.0)), L["RK4"], "RK4"),
         "rk4big": ("RK4, 허수축 구간보다 큰 걸음", rk4, int(np.ceil(T * rho / 3.2)), L["RK4"], "RK4"),
         "ssp": ("SSP-RK3", ssprk3, int(np.ceil(T * rho / 1.5)), L["SSP-RK3"], "SSP-RK3")}
for key in sys.argv[1:]:
    nm, st, N, cl, cn = cases[key]; t0 = time.time()
    rep = verify(S, step=st, N=N, claim=cl, claim_name=cn, eps=1e-3, name=nm)
    print(format_report(rep, title=f"[{nm}] N = {N}  ({time.time() - t0:.1f}초)"), flush=True); print()
```

작동 확인 space_bugs.py — A6·4.5절: 공간 스킴의 에러 검사(1–3차원, 시간 먼저, 비선형, 블랙박스, 일부러 넣은 버그)

```
"""공간 이산의 에러 검사(space_check)를 맞는 구현과 일부러 틀린 구현에 돌린다.
각 사례: 대상 코드가 만든 부동소수 행렬(또는 블랙박스 함수) L, 마디 위치 x, 주장한 이산."""
import sys, numpy as np
sys.path.insert(0, "../numide")
from operators import fdm_vertex, fvm_cell, fem_p1, fem_p1_graded
import space_check as SC
from fractions import Fraction as Fr
from six_w import format_report


def p1_2d(xs, ys, skip=None, badcot=None):
    """대상 코드(2차원 P1, 집중 질량): 각 사각형을 (x_j, y_i)-(x_{j+1}, y_{i+1}) 대각선으로 나누고 기울기로 요소 강성을 조립.
    skip = (i, j, t): 그 삼각형을 조립하지 않는 버그;  badcot = (i, j): 그 사각형의 첫 삼각형에서 기울기 행렬을 틀리게 쓰는 버그"""
    n, m = len(ys), len(xs); N = n * m; K = np.zeros((N, N)); w = np.zeros(N); idx = lambda i, j: i * m + j
    for i in range(n - 1):
        for j in range(m - 1):
            P = {idx(i, j): (xs[j], ys[i]), idx(i, j + 1): (xs[j + 1], ys[i]), idx(i + 1, j): (xs[j], ys[i + 1]), idx(i + 1, j + 1): (xs[j + 1], ys[i + 1])}
            for t, ids in enumerate(((idx(i, j), idx(i, j + 1), idx(i + 1, j + 1)), (idx(i, j), idx(i + 1, j + 1), idx(i + 1, j)))):
                if skip == (i, j, t): continue
                Pm = np.array([P[k] for k in ids]); B = np.array([Pm[1] - Pm[0], Pm[2] - Pm[0]]).T; area = 0.5 * abs(np.linalg.det(B))
                G = np.linalg.inv(B).T @ np.array([[-1, 1, 0], [-1, 0, 1]], float)
                if badcot == (i, j) and t == 0: G = np.linalg.inv(B) @ np.array([[-1, 1, 0], [-1, 0, 1]], float)   # 전치를 빠뜨림
                Ke = area * G.T @ G
                for a in range(3):
                    w[ids[a]] += area / 3
                    for b in range(3): K[ids[a], ids[b]] += Ke[a, b]
    return -K / w[:, None]


def case(name, L, x, kind, note=""):
    r = SC.check(L, x, kind)
    fix = SC.check(SC.patched(SC.as_matrix(L, len(x) if not isinstance(x, tuple) else int(np.prod([len(a) for a in x]))), r["viol"]), x, kind) if not r["ok"] else None
```

```
    rep = SC.report(r, x, name, fix)
    print(format_report(rep, title=f"[{name}]" + (f"  — {note}" if note else """)), flush=True); print()
    return r


L, w, x = fdm_vertex(21, 1.0); h = x[1] - x[0]
case("FDM(꼭짓점) 21마디", L, x, "FDM")
Lb = L.copy(); Lb[0, 1] = 1.0 / h ** 2
case("FDM 경계 버그", Lb, x, "FDM", "유령점의 2배를 빠뜨림: L[0,1] = 1/h²")
Lb = L.copy(); Lb[10, 10] *= 1.01
case("FDM 내부 버그", Lb, x, "FDM", "마디 10의 대각에 1% 오타")
L, w, x = fvm_cell(20, 1.0); h = x[1] - x[0]
case("FVM(칸 중심) 20칸", L, x, "FVM")
Lb = L.copy(); Lb[7] /= 2
case("FVM 칸 너비 버그", Lb, x, "FVM", "칸 7을 2h로 나눔(행합·1차 보존은 그대로)")
L, w, x = fem_p1_graded(21, 1.0)
case("FEM P1 비균일 21마디", L, x, "FEM P1")
hh = np.diff(x); K = np.zeros((21, 21))
for e in range(20): K[e:e + 2, e:e + 2] += np.array([[1, -1], [-1, 1]]) / hh[e]
wb = np.append(hh, hh[-1])
case("FEM P1 집중 질량 버그", -K / wb[:, None], x, "FEM P1", "집중 질량을 (h_{i-1} + h_i)/2 대신 h_i로 씀")
K2 = K.copy(); K2[5:7, 5:7] -= np.array([[1, -1], [-1, 1]]) / hh[5]; m2 = w
case("FEM P1 조립 버그", -K2 / m2[:, None], x, "FEM P1", "요소 5(마디 5–6)의 강성을 조립하지 않음")
xs = np.linspace(0, 1, 6); ys = np.linspace(0, 1, 6)
case("2차원 P1 삼각형 6×6", p1_2d(xs, ys), (xs, ys), "FEM P1 삼각형")
case("2차원 P1 기울기 버그", p1_2d(xs, ys, badcot=(2, 3)), (xs, ys), "FEM P1 삼각형", "사각형 (2, 3)의 한 삼각형에서 기울기 행렬의 전치를 빠뜨림")
case("2차원 P1 조립 버그", p1_2d(xs, ys, skip=(4, 1, 1)), (xs, ys), "FEM P1 삼각형", "사각형 (4, 1)의 둘째 삼각형을 조립하지 않음")
# 블랙박스: 대상은 u -> L u 함수만 준다(행렬은 단위 벡터로 뽑는다)
L, w, x = fdm_vertex(21, 1.0)
def apply_bug(u):
    v = L @ u; v[15] += 0.5 * (u[16] - u[15]) / (x[1] - x[0]) ** 2; return v           # 마디 15의 오른쪽 결합을 1.5배로
case("블랙박스 FDM(함수만)", apply_bug, x, "FDM", "u -> L u 함수 안에서 마디 15의 오른쪽 결합이 1.5배")
# 주장 없이(규칙 R2–R3만): 조립을 빠뜨린 FEM은 주장 없이도 잡히고, 칸 너비 버그는 주장이 있어야 잡힌다
L, w, x = fem_p1_graded(21, 1.0)
r = SC.check(-K2 / m2[:, None], x, None); print("[주장 없이] FEM P1 조립 버그:", "반례 " + ", ".join(f"마디 {v['i']} {v['rule']}" for v in r["viol"]) if not r["ok"] else "통과"); print()
L, w, x = fvm_cell(20, 1.0); Lb = L.copy(); Lb[7] /= 2
r = SC.check(Lb, x, None); print("[주장 없이] FVM 칸 너비 버그:", "반례" if not r["ok"] else "통과(행합과 1차 보존은 지켜서 주장 없이는 잡히지 않는다)"); print()
# 시간 먼저(로테), 걸음마다 격자가 바뀜: 층마다 공간 연산자, 층 사이 보간
grids = [np.linspace(0, 1, 11), fem_p1_graded(11, 1.0, 0.3)[2], fem_p1_graded(13, 1.0, 0.5)[2]]
def interp(xa, xb, bug=False):
    P = np.zeros((len(xb), len(xa)))
    for a_, y in enumerate(xb):
        i = min(np.searchsorted(xa, y, side="right") - 1, len(xa) - 2); s = (y - xa[i]) / (xa[i + 1] - xa[i])
        if bug: s = 1 - s                                  # 보간 무게를 거꾸로
        P[a_, i] += 1 - s; P[a_, i + 1] += s
    return P
def layers(bug, fixv=None):
    allv = []
    for k, xg in enumerate(grids):
        allv += SC.check(fem_p1(xg)[0], xg, "FEM P1", level=k)["viol"]
        if k > 0:
            P = interp(grids[k - 1], xg, bug=(bug and k == 2))
            if fixv: P = SC.patched(P, [v for v in fixv if v["level"] == k], rule="보간 관계식(P1)")
            allv += SC.check_interp(P, grids[k - 1], xg, level=k)["viol"]
    return dict(ok=not allv, viol=allv, n=sum(len(g) for g in grids), rules="관계식, 상수 보존, 1차 보존, 보간", kind="FEM P1(층마다)")
for bug in (False, True):
    res = layers(bug); fix = layers(bug, res["viol"]) if not res["ok"] else None
    print(format_report(SC.report(res, grids[2], "시간 먼저 3층", fix), title="[시간 먼저(로테), 층마다 격자" + (", 2층 보간 버그]" if bug else "]")), flush=True); print()


# ---------------------------------------------------------------------- 임의 차원: 3차원 텐서 격자(축별 1차원 이산의 크로네커 합, 대상 코드)
def kron_sum(Ls):
    """대상 코드: L = sum_k I ⊗ ... ⊗ L_k ⊗ ... ⊗ I (축 0이 가장 빨리 도는 번호)"""
    d = len(Ls); n = [A.shape[0] for A in Ls]; out = 0
    for k in range(d):
        M = np.eye(1)
        for l in reversed(range(d)): M = np.kron(M, Ls[l] if l == k else np.eye(n[l]))
        out = out + M
    return out
ax3 = [np.linspace(0, 1, 5), np.linspace(0, 0.5, 4), np.linspace(0, 2, 6)]
```